

InterpLens Granular Phase Execution Plan

Design Principle: Every phase is built, optimized (CUDA / PyTorch memory management), verified, and tested step-by-step before moving to the next.

Phase Overview

```
Phase 1: High-Performance Engine & Base Adapter (CUDA Memory Safe)
↓
Phase 2: Logit Lens & Micro-FastAPI Engine (First End-to-End Vertical Slice)
↓
Phase 3: Residual Stream & Attention Head Engines
↓
Phase 4: Neuron Activation & Token Attribution Engines
↓
Phase 5: Automated Causal Patching & Causal Tracing (ROME Sweeps)
↓
Phase 6: Multi-Model Family Adapters & Model Comparison Diffing
↓
Phase 7: Sparse Autoencoder (SAE) Feature Explorer
↓
Phase 8: Polish, UI Bundling & PyPI Distribution
```

CUDA & PyTorch Optimization Mandate for All Phases

Across every phase, all backend tensor operations MUST adhere to these PyTorch & CUDA best practices:

1. `@torch.inference_mode()`: Used for all forward passes and interpretability projections (bypasses autograd graph overhead).
2. **Mixed Precision** (`bfloat16` / `float16`): Native support for half-precision inference to halve VRAM usage on Modern GPUs (Ampere/Hopper/Ada).
3. **Zero-Copy Tensor Slicing**: Avoid `.clone()` or redundant memory allocations where read-only views suffice.
4. **CUDA Memory Tracking & Cleanup**: Explicit VRAM cache clearing (`torch.cuda.empty_cache()`) during session drops.
5. **Vectorized Math**: Compute unembedding projections ($X \cdot W_U$) across all layers in single matrix multiplications rather than Python loops over layers.

Detailed Phase Breakdowns

Phase 1: High-Performance Engine & Base Adapter Layer [COMPLETED - 100% Passed]

Goal: Establish the foundational Python package structure, PyTorch/CUDA optimization settings, abstract adapter interfaces, zero-copy in-memory model attachment, and LRU session caching.

- [x] **1.1 Package Skeleton:** Built `pyproject.toml`, `interplens/__init__.py`, `interplens/config.py`, `interplens/schema.py`.
- [x] **1.2 CUDA / PyTorch Config:** Implemented memory utilities (`get_optimal_device()` , VRAM monitoring, `free_gpu_memory()`).
- [x] **1.3 BaseModelAdapter Abstract Interface:** Defined standard methods for tokenization, `@torch.inference_mode()` forward passes, and layer hook resolution.
- [x] **1.4 InPlaceModelAdapter:** Implemented zero-copy wrapper for existing `HookedTransformer` or `nn.Module` objects already in VRAM.
- [x] **1.4b CustomModelAdapter & PyTorchAutoHooker:** Supported novel/custom PyTorch `nn.Module` architectures via native `register_forward_hook()` interception.
- [x] **1.5 LRU Session Store:** Implemented thread-safe `SessionStore` with VRAM tracking and automatic eviction of old activation caches.
- [x] **1.6 Phase 1 Verification:** Ran automated unit tests (`pytest tests/test_phase1.py`) -> 5/5 tests passed (100%).

Phase 2: Logit Lens & Micro-FastAPI Engine (NEXT STEP)

Goal: Prove the full stack end-to-end (Engine -> Adapter -> FastAPI -> Web UI) using the Logit Lens feature.

- [] **2.1 Logit Lens Engine:** Write vectorized unembedding algorithm (`interplens/analysis/logit_lens.py`).
- [] **2.2 FastAPI Backend Setup:** Implement `/api/health`, `/api/run`, and `/api/analysis/logit-lens`.
- [] **2.3 Minimal Web UI:** Embedded dashboard with prompt input, layer slider, and interactive Logit Lens table & ribbon plot.
- [] **2.4 Phase 2 Verification:** End-to-end test with a real prompt on `gpt2` verifying top token predictions across layers.

Phase 3: Residual Stream & Attention Head Engines

Goal: Add deep spatial internal views for representations and attention weights.

- [] **3.1 Residual Stream Engine:** Vectorized L2 norms, variances, and 2D PCA trajectory computation (`residual_stream.py`).

- [] **3.2 Attention Head Engine:** Vectorized extraction of $N \times N$ attention matrices and arc diagram link data (`attention_heads.py`).
 - [] **3.3 UI Panel Integration:** Add Residual Stream Heatmap / PCA tab and Attention Matrix Grid / Arc Diagram tab.
 - [] **3.4 Phase 3 Verification:** Benchmark tensor extraction speed and UI rendering latency.
-

🟡 Phase 4: Neuron Activations & Token Attribution Engines

Goal: Provide fine-grained component attribution (which MLP neurons fired and which input tokens mattered).

- [] **4.1 Neuron Engine:** Rank top-K firing neurons per token and generate prompt text lighting strips (`neurons.py`).
 - [] **4.2 Token Attribution Engine:** Implement gradient and rollout token attribution (`attribution.py`).
 - [] **4.3 UI Panel Integration:** Add Neuron Bar Chart + Text Strips tab and Token Attribution Highlight tab.
 - [] **4.4 Phase 4 Verification:** Unit tests for top-K sorting correctness and non-zero attributions.
-

🔴 Phase 5: Automated Causal Interventions (Patching & ROME Sweeps)

Goal: Enable causal experimentation (moving beyond correlation to causal proof).

- [] **5.1 Activation Patching Engine:** Swap activations between clean & corrupt prompts (`causal_patching.py`).
 - [] **5.2 Causal Tracing Engine:** Automated layer $\times$ position sweep generator (`causal_tracing.py`).
 - [] **5.3 UI Panel Integration:** Interactive Causal Patching Heatmap tab with clean/corrupt prompt inputs and single-cell drilldown.
 - [] **5.4 Phase 5 Verification:** Verify patching logit-diff recovery on known benchmark prompts.
-

🟠 Phase 6: Multi-Model Family Expansion & Model Comparison Engine

Goal: Expand beyond GPT-2 to Llama-3.2 and Qwen2.5, and enable side-by-side model diffing.

- [] **6.1 Model Adapters:** Implement `GPT2Adapter` , `QwenAdapter` , `LlamaAdapter` .
 - [] **6.2 Model Comparison Engine:** Synchronized dual-model execution and tensor diffing ($\$Model_A - Model_B$).
 - [] **6.3 UI Panel Integration:** Model Comparison split-screen and Diff heatmap tab.
 - [] **6.4 Phase 6 Verification:** Verify cross-family adapter parity.
-

● Phase 7: Sparse Autoencoder (SAE) Feature Explorer

Goal: Decompose polysemantic neurons into monosemantic sparse features using pretrained SAEs.

- [] **7.1 SAE Loader:** Integrate public pretrained SAE loading (Neuronpedia / EleutherAI).
 - [] **7.2 Feature Activation Engine:** Compute feature activations and label lookups (`sae_explorer.py`).
 - [] **7.3 UI Panel Integration:** SAE Feature Explorer panel with label search and activation strips.
-

○ Phase 8: Polish, Packaging & PyPI Distribution

Goal: Finalize distribution readiness and developer experience.

- [] **8.1 UI Static Bundle:** Build static HTML/JS/CSS bundle into `interplens/ui/dist` .
- [] **8.2 PyPI Build:** Test wheel build (`python -m build`) and `pip install .` .
- [] **8.3 Documentation & Notebook Examples:** Ship user guide and Jupyter notebook tutorials.